

Minesweeper Clone

EECS 581, Group #7

Team Members: Sean Crosby, Thad Flourney, Eyassu Mongalo, Lily Brooks-Kanost, Caleb Neil, Nick Paslay

1. Introduction

1.1. Overview

This program is a recreation of the popular desktop game Minesweeper, a grid-based strategy game where players must find and mark hidden bombs. The game is one-player and has no limit on game or turn time, so a user may take as long as they need to solve the puzzle. This version of Minesweeper was built in C++ using GTK for graphics

1.2. Game Rules

- Click a box in the 10x10 grid to start the game.
- Clicking a box will either reveal a bomb, causing you to lose the game, or a number which is the amount of bombs that are touching that box.
- You may right click an unclicked box to flag the box. Flagging a box is usually for a reminder that a bomb is more than likely there. Only implemented for one's convenience, there is no effect on the game to use or not use a flag.

1.3. Hours Invested

- Scrum Master - Nick Paslay (3-4 hours)
- UI / Graphics -Lily Brooks-Kanost/Caleb Neil (4-5 hours)
- Gameplay Engineer -Eyassu Mangola (5-6 hours)
- Input Handler - Thad Flourney (3-4 hours)
- Board Generation - Sean Crosby (4-5 hours)

1.4. Team Organization

- Scrum Master - Nick Paslay
- UI / Graphics -
- Gameplay Engineer -
- Input Handler - Thad Flourney
- Board Generation - Sean Crosby

2. System Overview

- System Description: Minesweeper is a desktop game designed in C++ using GTK graphics for a graphical interface. The game lets the user choose a tile on a 10x10 grid that will generate a 3x3 grid of nonbomb tiles with the intention to clear the whole grid by choosing all other tiles that do not contain the bomb. The game will indicate whether a bomb is touching the tile either directly or diagonally and will indicate how many bombs are touching that tile.
- Design Goals: To create a playable minesweeper game implemented with GTK graphics for interface user convenience.
- File Structure: source files will contain C++ code and include files will contain header files for declaration of functions and classes.

3. Architectural Design

- Classes:
 - Tile
 - Board
 - ClickType
 - TileAction
 - TileChange
 - InputHandler
 - UIWindow
 - StartWindow
 - GameWindow

4. Detailed Design

Board.cpp

Board::initialize

- Responsibilities: Create a 2d array representing a 10x10 Minesweeper board, complete with bombs and marked tiles
- Interface:
 - Inputs: board_size, bomb_count, x and y (initial tile clicked)
 - Outputs: instance of Board class
 - Error Handling: Potential errors mostly involve indexing non-existent array values. Whenever referencing the array, strict index limits are checked as to avoid these errors
- Description:
 - After creating a blank instance of the Board class filled with default tiles, the function enters a while loop conditional on n_bombs equal to zero. Each loop, the function will attempt to place a bomb in a random location. If the location accessed is within a 3x3 “safe zone” of the initial revealed tile, then the loop is restarted and a new value is created. If the tile is outside of the safe zone, then the tile’s value is set to 9, and the Board::update_tile_values function is called, passing in this tile as input.

Board::update_tile_values

- Responsibilities: Update tile values within a 3x3 area of a placed bomb
- Interface:
 - Inputs: x and y (bomb location)
 - Outputs: none
 - Error Handling: Potential errors mostly involve indexing non-existent array values. Whenever referencing the array, strict index limits are checked as to avoid these errors
- Description:
 - First, limits for the updates are set by checking the 3x3 area around the initial x and y input.

Board::is_bomb

- Responsibilities: Return whether not a given index is a bomb
- Interface:
 - Inputs: x and y (map location)
 - Outputs: boolean whether or not the location is a bomb
 - Error Handling: Errors will only happen if given x & y values outside of the range
- Description:
 - Returns true if the tile @ location (x, y) is a bomb

Board::is_empty

- Responsibilities: Return whether or not a given index has bombs around it
- Interface:
 - Inputs: x and y (map location)
 - Outputs: boolean whether surrounding indices have a bomb
 - Error Handling: Errors will only happen if given x & y values outside of the range
- Description:
 - Returns true if the tile @ location (x, y) has bombs in the neighboring coordinates

Board::get_state

- Responsibilities: Update tile values within a 3x3 area of a placed bomb
- Interface:
 - Inputs: x and y (map location)
 - Outputs: Tile::TileState, which can be Uncovered, Covered, or Flagged
 - Error Handling: Errors will only happen if given x & y values outside of the range
- Description:
 - Returns the tile state @ location (x, y)

Board::uncover_surrounding

- Responsibilities: Uncover tile values within a 3x3
- Interface:
 - Inputs: x and y (map location)
 - Outputs: None
 - Error Handling: Errors
- Description:
 - Uncover tiles within a 3 by 3 area, recursively calling itself if any of the tiles are empty

Board::uncover

- Responsibilities: Uncover tile values within a 3x3
- Interface:
 - Inputs: x and y (map location)
 - Outputs: returns true if a bomb was uncovered
 - Error Handling: Errors will only happen if given x & y values outside of the range
- Description:
 - Uncover tiles recursively calling itself if the tiles @ location (x, y) are empty

Input_handler.cpp

InputHandler::InputHandler

- Responsibilities: Initializes the input handler with the board size
- Interfaces/APIs:
 - Inputs: int board_size
 - Outputs: Constructs an input handler object
 - Error Handling: The caller should provide a positive board size
- Description:
 - Stores the board size so button IDs can be converted to row and column coordinates.

InputHandler::get_tile

Responsibilities: Converts a button ID to board coordinates

- Interfaces/APIs:
 - Inputs: int button_id
 - Outputs: std::pair<int, int> with row, col
 - Error Handling: The caller should provide an ID within the boards range
- Description:
 - Converts a button ID into a row and column pair. Buttons are assumed to be assigned left to right and top to bottom.

InputHandler::handle_tile_click

- Responsibilities: Process left and right clicks
- Interfaces/APIs:
 - Inputs: int button_id, ClickType type (LEFT or RIGHT)
 - Outputs: vector containing all the tile changes caused by the click
 - Error Handling: Returns an empty vector if the appropriate callback is not assigned.
- Description:
 - Converts the button ID into coordinates and calls revealing the tile or flagging the tile, returns all tile changes caused by the action

InputHandler::on_reveal_tile

- Responsibilities: Provides the callback for revealing a tile
- Interfaces/APIs:
 - Inputs: int row, int col
 - Outputs: vector describing the tiles revealed
 - Error Handling: The callback owner is responsible for validating coordinates and handling invalid or disallowed reveals
- Description:
 - Stores the function that should run when a tile is left-clicked.

InputHandler::on_flag_tile

Responsibilities: Provide the callback for flagging a tile

- Interfaces/APIs:
 - Inputs: int row, int col
 - Outputs: vector describing the flag or unflag action
 - Error Handling: The callback owner should validate coordinates and handle invalid or disallowed flag actions.
- Description:
 - Stores the function that should run when a tile is right-clicked.

InputHandler::on_start_game, on_retry, on_close

- Responsibilities: Store callbacks for game lifecycle actions
- Interfaces/APIs:
 - Inputs: int num_bombs, none
 - Outputs: None
 - Error Handling: The callback owner is responsible for validating bomb count, also handling retry or close behavior.
- Description:
 - These callbacks allow the UI to trigger game start, retry, and close behavior.

Ui_manager.cpp

StartWindow::StartWindow()

- Responsibilities: Allows popup window with the board that was created, allows for configuration of bombs, the task of displaying the start game button.
- Interfaces/APIs:
 - Outputs: GTK container
- Description: simply initializes GTK graphical interface

StartWindow::on_start_button_clicked() and Start_game(int bomb count)

- Responsibilities: handling the start button and the GTK graphical interface.
- Interfaces/APIs:
 - Inputs: bomb slider selection and request of start game
 - Outputs: board to be generated on click
- Description: this function retrieves the bomb count from the slider and distributes it to the board generation function for when a tile is clicked, it will create the requested number of bombs on board generation.

GameWindow::GameWindow() and ~GameWindow()

- Responsibilities: initializes main minesweeper game window with input callbacks and tile button handling. ~GameWindow will deconstruct when needed.
- Interfaces/APIs:
 - Outputs: produces a fully initialized minesweeper game
- Description: creates the main graphical interface for minesweeper.

reset_game()

- Responsibilities: Resets the game with the existing bomb count.
- Interfaces/APIs:
 - Inputs: uses bomb count.
 - Outputs: calls start_game and resets the game
- Description: resets the game to the start game board with the same amount of bombs that were requested by the slider.

GameWindow::update_button_display(int row, int col)

- Responsibilities: updates the tile to match the current board state.
- Interfaces/APIs:
 - Inputs: row, col
 - Outputs: updates the appearance and behavior button to whatever is needed
- Description: this function will display what the board needs to display (flags for a flagged tile, bomb for a bomb tile, number for the open tile)

reveal_tile(int row, int col)

- Responsibilities: gives the behavior of the tile after being clicked by a left mouse input.
- Interfaces/APIs:
 - Inputs: row and col
 - Outputs: a loose screen, a mine hit, a revealing of the whole map, or a tile reveal.
- Description: this is a needed function to call other functions when the left mouse button clicks a tile

flag_tile(int row, int col)

- Responsibilities: places or removes a flag
- Interfaces/APIs:
 - Inputs: row and col
 - Outputs: returns either a flag or unflag action
- Description: places or removes a flag

On_button_clicked and on_button_right_clicked()

- Responsibilities: in charge of left and right clicked mouse inputs
- Interfaces/APIs:
 - Inputs: left or right click of mouse input
 - Outputs: callbacks or updates to game interface(flag place, tile reveal) after input.
- Description: handles all left and right mouse input events and displays results of mouse input.

Show_end_screen()

- Responsibilities: displays end screen
- Interfaces/APIs:
 - Inputs: won ,true, false
 - Outputs: will display either winner or defeat
- Description: the end screen to the graphical interface that will display a win or lose and the option to play again or quit.

on_play_again_clicked() and on_quit_clicked()

- Responsibilities: either reloads the game again or quits the game
- Interfaces/APIs:
 - Inputs: mouse input
 - Outputs: closes game or reloads board
- Description: depending on the choice of the user, the game with either reload with a new board to play on, or will quit the graphical interface and end the program.

5. Performance and Scalability

- Expected Load: A singular user to play the game at a time.
- Optimizations: This code is meant to suffice one user to choose the amount of bombs on a 10x10 grid. Only the tile that is chosen with mouse input is updated on the graphics interface. The board will generate at the start of the game once the user has chosen a tile to begin, restricting constant graphic change.
- Limitations: This program is only meant for a restricted 10x10 grid size, with no scalability for choosing a size of board, and meant for 10-20 bombs on the board.

6. Program Environment

- Environments: Minesweeper will work willingly on Linux, and requires package downloads to run on windows. Unfortunately, this minesweeper will not work on macOS.
- Required Packages: WINDOWS SPECIFIC: g++, make, gtkmm-4.0, pkg-config.

7. Testing Strategy

- Unit Testing: we tested the start and end game screen, bombs, flags, tiles with a right click for flag, left click for choosing a tile, all separately as we wrote the code.
- Integration Testing: periodic game logic testing was needed, as we began creating graphics. We implemented the graphics and tested beginning board generation, then a bomb tile, and normal tile separately.
- End-to-End Testing: We testing launching the application, the end game screen, the start screen, changing numbers of bombs and loading the board all separately and then all together
- Quality Metrics: Another member of the team had to review and merge the branch of a member when coding, and test cases were used to credit error codes and consistency.